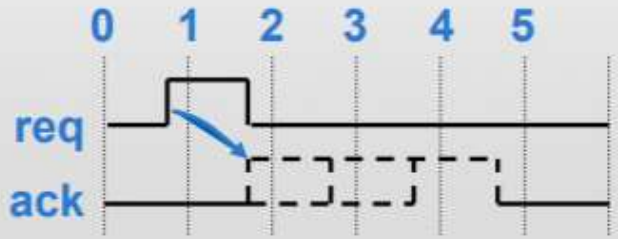
What Is An Assertion?

- **An assertion is a statement that a certain property must be true**



Design Specification:
After the request signal is asserted, the acknowledge signal must arrive 1 to 3 clocks later

- **Assertions are used to:**
  - Document design intent (e.g.: every request has an acknowledge)
  - Verify design meets the specification over simulation time
  - Verify design assumptions (e.g.: state value is one-hot)
  - Localize where failures occur in the design instead of at the output
  - Provide semantics for formal verification
  - Describe functional coverage points
  - And… requires clarifying ambiguities in spec

Which one of these is the most important to you in your projects?

accellera

SystemVerilog Assertions

- Assertions are primarily used to validate the behavior of a design.
- An assertion is a check embedded in design or bound to a design unit during the simulation.
- Warnings or errors are generated on the failure of a specific condition or sequence of events.

Assertions are used to,

- Check the occurrence of a specific condition or sequence of events.
- Provide functional coverage.

There are two kinds of assertions:

- Immediate Assertions
- Concurrent Assertions

Immediate Assertions:

- *Immediate assertions* check for a condition at the current simulation time.
- An *immediate assertion* is the same as an if..else statement with assertion control.
- Immediate assertions have to be placed in a procedural block definition.

Syntax

```
label: assert(expression) action_block;
```

- *The optional statement label (identifier and colon) creates a named block around the assertion statement*
- *The action block is executed immediately after the evaluation of the assert expression*
- *The action_block specifies what actions are taken upon success or failure of the assertion*

action_block;

```
pass_statement; else fail_statement;
```

•The **pass statement** is executed if the expression evaluates to true
•The statement associated with else is called a **fail statement** and is executed if the expression evaluates to false
•Both pass and fail statements are **optional**
•Since the assertion is a statement that something must be true, the failure of an assertion shall have a severity associated with it.
•By **default**, the severity of an assertion failure is an **error**.
•Other severity levels can be specified by including one of the following severity system tasks in the fail statement:

- $fatal is a run-time fatal.
- $error is a run-time error.
- $warning is a run-time warning, which can be suppressed in a tool-specific manner.
- $info indicates that the assertion failure carries no specific severity.

•If an assertion fails and no else clause is specified, the tool shall, by default call $error.

```
//With Pass and Fail statement; Fail verbosity info;
assert(expression) $display("expression evaluates to true"); else $display("expression
evaluates to false");

//Only With Pass statement;
assert(expression) $display("expression evaluates to true");

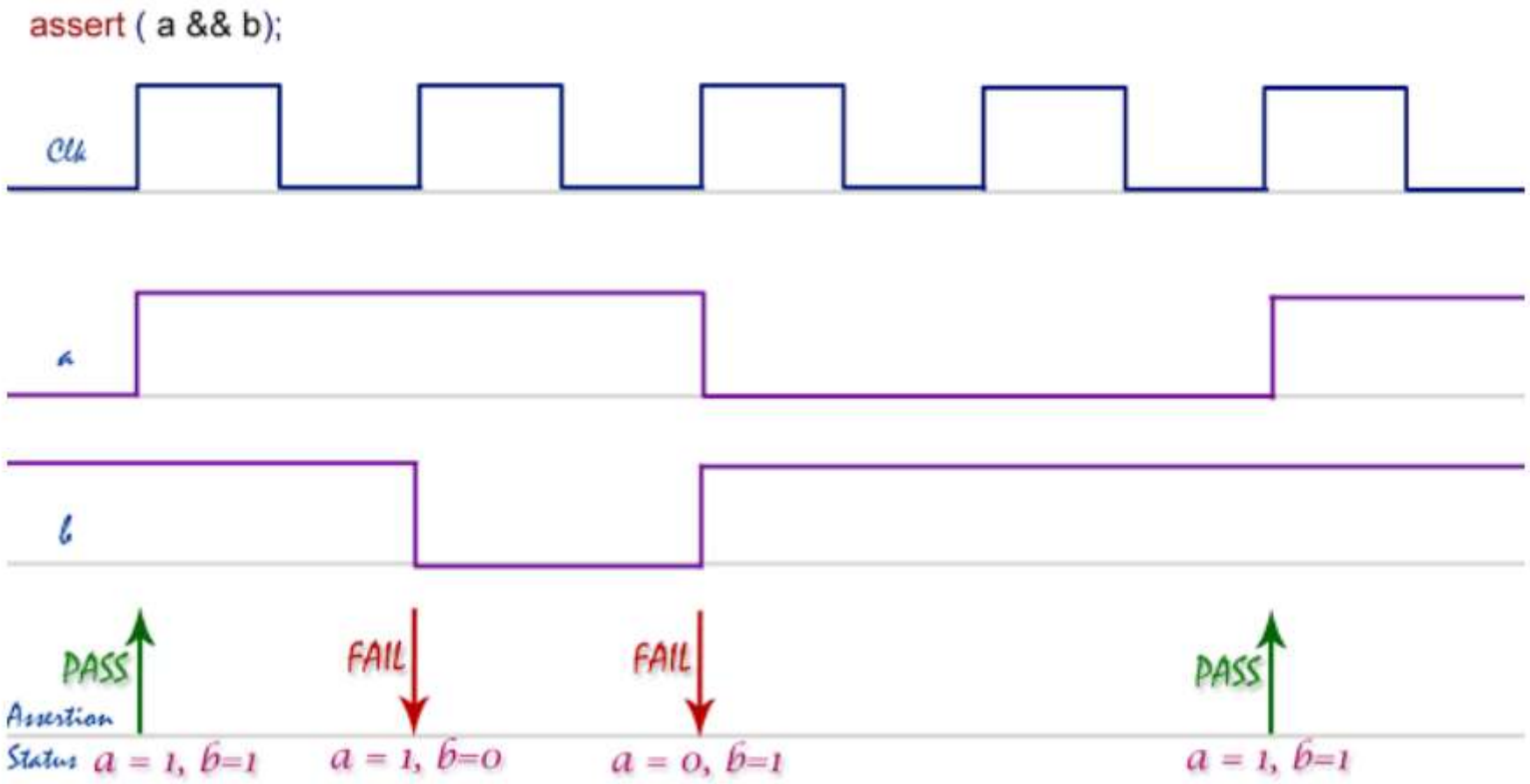
//With Pass and Fail statement; Fail verbosity fatal;
assert(expression) $display("expression evaluates to true"); else $fatal("expression
evaluates to false");

//Only With Fail statement; Multiple statements in Faile condition and Fail verbosity fatal;
assert(expression)
      else begin
        …….
        …….
        $fatal("expression evaluates to false");
      end

//Only With Fail statement; Fail verbosity warning;
assert(expression) else $warning("expression evaluates to false");

//With Label and Fail statement; Fail verbosity warning;
label: assert(expression) else $warning("expression evaluates to false");
```

```
always @(posedge clk) assert (a && b);
```



```
module asertion_ex;
  bit clk,a,b;

  //clock generation
  always #5 clk = ~clk;

  //generating 'a'
  initial begin
    a=1;
    b=1;
    #15 b=0;
    #10 b=1;
        a=0;
    #20 a=1;
    #10;
    $finish;
  end

  //Immediate assertion
  always @(posedge clk) assert (a && b);

endmodule
```

```
ncsim: *E,ASRTST (./testbench.sv,23): (time 15 NS) Assertion asertion_ex.__assert_1 has failed
ncsim: *E,ASRTST (./testbench.sv,23): (time 25 NS) Assertion asertion_ex.__assert_1 has failed
ncsim: *E,ASRTST (./testbench.sv,23): (time 35 NS) Assertion asertion_ex.__assert_1 has failed
Simulation complete via $finish(1) at time 55 NS + 0
```

Concurrent Assertions:

Concurrent assertions check the sequence of events spread over multiple clock cycles.

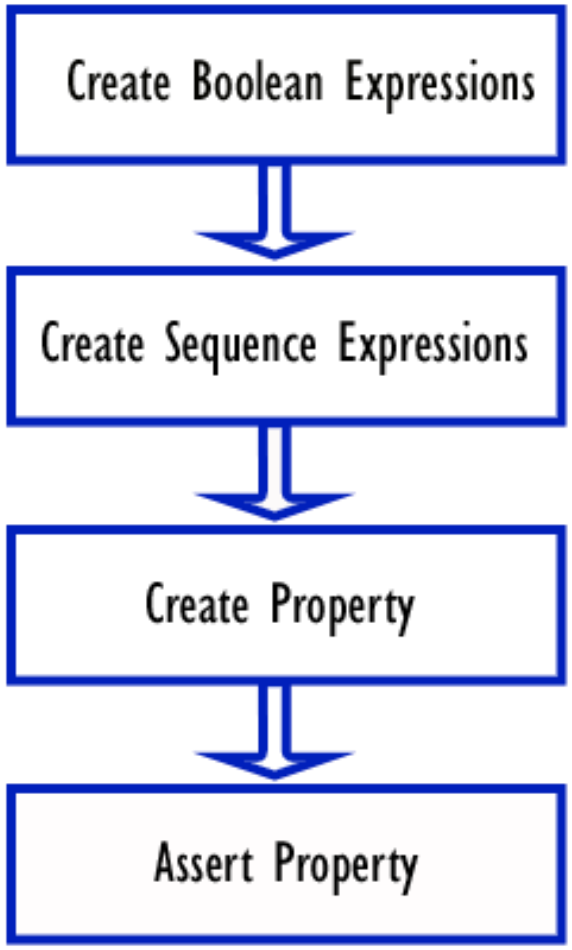
- The concurrent assertion is evaluated only at the occurrence of a clock tick
- The test expression is evaluated at clock edges based on the sampled values of the variables involved
- It can be placed in a procedural block, a module, an interface or a program definition

```
c_assert: assert property(@(posedge clk) not(a && b));
```

The Keyword differentiates the immediate assertion from the concurrent assertion is "property."

Building blocks of SVA

Below diagram shows the steps involved in the creation of an SVA checker,



```
if (A == B) ... // Simply checks if A equals B
assert (A == B); // Asserts that A equals B; if not, an error is generated
```

"The Read and Write signals should never be asserted together."

```
assert property ( !(Read && Write) );
```

"A Request should be followed by an Acknowledge occurring no more than two clocks after the Request is asserted."

```
assert property ( @(posedge Clock) Req |-> ##[1:2] Ack);
```

Boolean expressions

The functionality is represented by the combination of multiple logical events. These events could be simple Boolean expressions.

Sequence

Boolean expression events that evaluate over a period of time involving single/multiple clock cycles. SVA provides a keyword to represent these events called "sequence."

Syntax

```
sequence name_of_sequence;
  ......
endsequence
```

Property

- A number of sequences can be combined logically or sequentially to create more complex sequences.
- SVA provides a keyword to represent these complex sequential behaviors called "property".

Syntax

```
property name_of_property;
  test expression or
  complex sequence expressions
endproperty
```

Assert

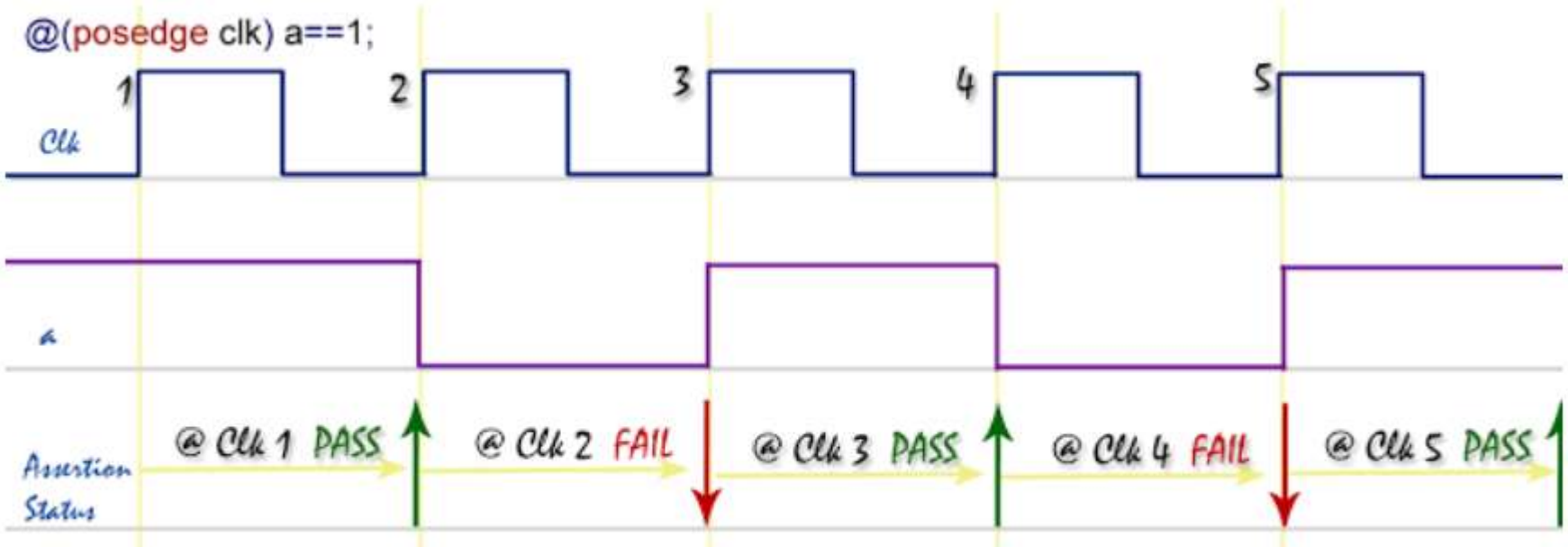
- The property is the one that is verified during a simulation.
- It has to be asserted to take effect during a simulation. SVA provides a keyword called "assert" to check the property.

Syntax

```
assertion_ name: assert_property( property_name );
```

SVA Sequence example

```
sequence seq_1;
  @(posedge clk) a==1;
endsequence
```



sequence seq_1 checks that the signal "a" is high on every positive edge of the clock. If the signal "a" is not high on any positive clock edge, the assertion will fail.

```
module asertion_ex;
  bit clk,a;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
    a=1;
    #15 a=0;
    #10 a=1;
    #10 a=0;
    #10 a=1;
    #10;
    $finish;
  end

  //assertion sequence
  sequence seq_1;
   @(posedge clk) a==1;
  endsequence

  //calling assert property
  a_1: assert property(seq_1);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

A sequence with a logical relationship

```
sequence seq_2;
  @(posedge clk) a || b;
endsequence
```

sequence, seq_2 checks that on every positive edge of the clock, either signal "a" or signal "b" is high. If both the signals are low, the assertion will fail.

Sequence Expressions

```
sequence seq_lib (a, b)
  a || b ;
endsequence
```

this seq can be used as,

```
sequence s_lib_inst
  seq_lib(req1,req2);
endsequence
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  //assertion sequence
  sequence seq_2;
    @(posedge clk) a||b;
  endsequence

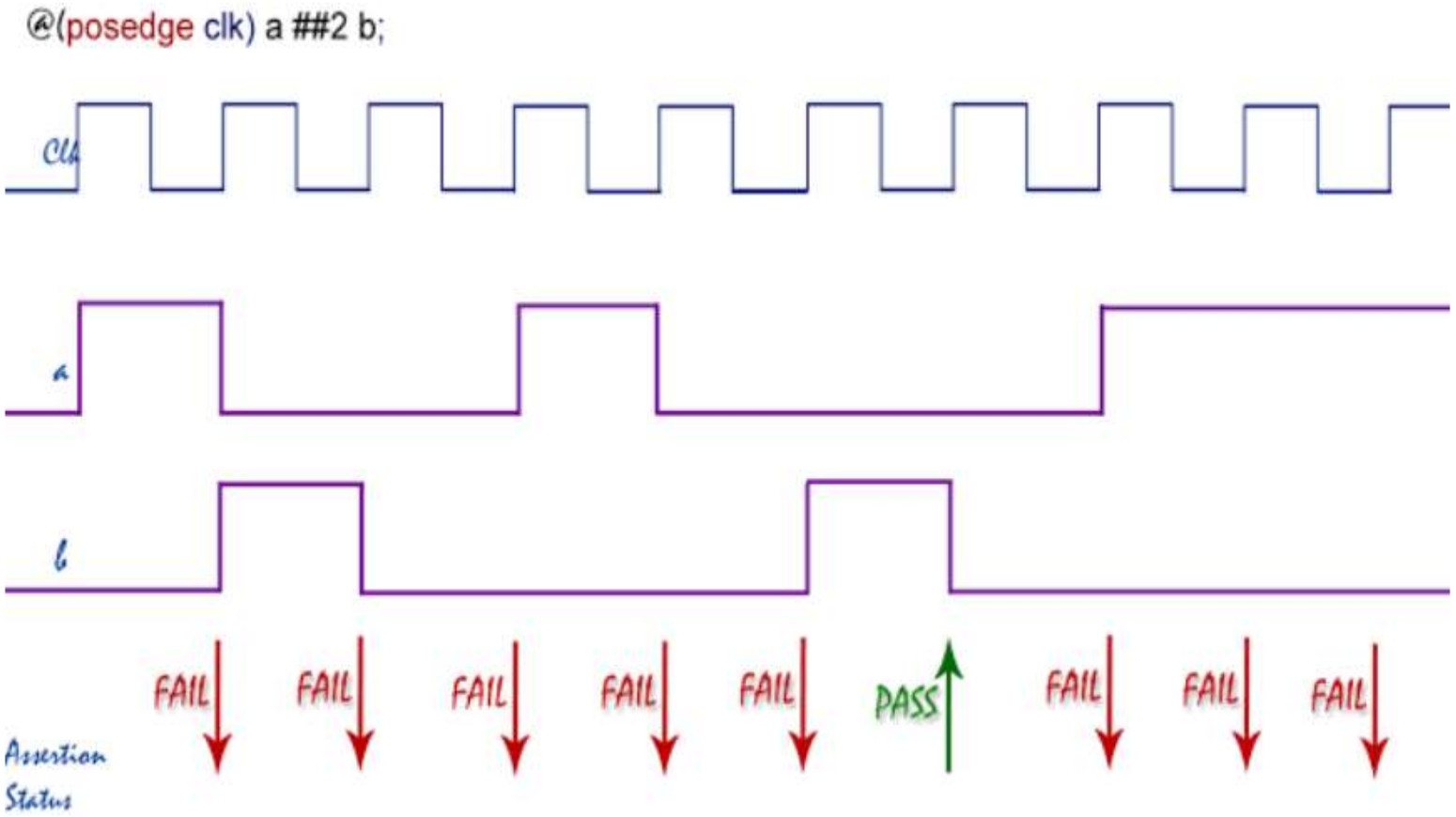
  //calling assert property
  a_1: assert property(seq_2);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

Sequences with timing relationship

In SVA, clock cycle delays are represented by a "##" sign. For example, ##2 means 2 clock cycles.

```
sequence seq;
  @(posedge clk) a ##2 b;
endsequence
```



sequence checks for the signal "a" being high on a given positive edge of the clock. If the signal "a" is not high, then the sequence fails. If signal "a" is high on any given positive edge of the clock, the signal "b" should be high 2 clock cycles after that. If signal "b" is not asserted after 2 clock cycles, the assertion fails.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  //assertion sequence
  sequence seq;
    @(posedge clk) a ##2 b;
  endsequence

  //calling assert property
  a_1: assert property(seq);

  //wave dump
```

```
Assertion 'a_1' FAILED at time: 25ns (3 clk), testbench.sv(26), scope: asertion_ex, start-time: 25ns (3 clk)
Assertion 'a_1' FAILED at time: 25ns (3 clk), testbench.sv(26), scope: asertion_ex, start-time: 5ns (1 clk)
Assertion 'a_1' FAILED at time: 35ns (4 clk), testbench.sv(26), scope: asertion_ex, start-time: 15ns (2 clk)
Assertion 'a_1' FAILED at time: 45ns (5 clk), testbench.sv(26), scope: asertion_ex, start-time: 45ns (5 clk)
```

Clock usage in SVA

Clock defined in the sequence definition

example shows the usage of a clock inside the sequence definition.

```
sequence seq;
  @(posedge clk) a ##2 b;
endsequence
```

Clock defined in the property definition

```
sequence seq;
  a ##2 b;
endsequence

property p;
  @(posedge clk) seq;
endproperty
a_1 : assert property(p);
```

- In general, it is a good idea to define the clocks in property definitions and keep the sequences independent of the clocks.
- This will help increase the re-use of the basic sequence definitions.

SystemVerilog Implication operator

```
sequence seq;
  @(posedge clk) a ##2 b;
endsequence
```

- In the above sequence, we can observe that sequence starts on every positive edge of the clock and it looks for "a" to be high on every positive clock edge.
- If the signal "a" is not high on any given positive clock edge, an error is issued by the checker.
- If we want the sequence to be checked only after “a” is high, this can be achieved by using the implication operator.

- The implication is equivalent to an if-then structure.
- The left-hand side of the implication is called the "antecedent" and the right-hand side is called the "consequent.“
- The antecedent is the gating condition.
- If the antecedent succeeds, then the consequent is evaluated.

The implication construct can be used only with property definitions. It cannot be used in sequences.

- There are 2 types of implication:
- Overlapped implication
- Non-overlapped implication

Overlapped implication

- The overlapped implication is denoted by the symbol |->.
- If there is a match on the antecedent, then the consequent expression is evaluated in the same clock cycle.

```
property p;
  @(posedge clk) a |-> b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" should also be high on the same clock edge.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
Assertion 'a_1' FAILED at time: 35ns (4 clk), testbench.sv(25), scope: asertion_ex, start-time: 35ns (4 clk)
```

Non-overlapped implication

The non-overlapped implication is denoted by the symbol |=>
If there is a match on the antecedent, then the consequent expression is evaluated in the next clock cycle.

```
property p;
  @(posedge clk) a |=> b;
endproperty
a: assert property(p);
```

if signal "a" is high on a given positive clock edge, then signal "b" should be high on the next clock edge.

The implication with a fixed delay on the consequent

```
property p;
  @(posedge clk) a |-> ##2 b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" should be high after 2 clock cycles.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |=> b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
Assertion 'a_1' FAILED at time: 25ns (3 clk), testbench.sv(25), scope: asertion_ex, start-time: 15ns (2 clk)
Assertion 'a_1' FAILED at time: 45ns (5 clk), testbench.sv(25), scope: asertion_ex, start-time: 35ns (4 clk)
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##2 b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
Assertion 'a_1' FAILED at time: 35ns (4 clk), testbench.sv(25), scope: asertion_ex, start-time: 15ns (2 c
```

The implication with a sequence as an antecedent

```
sequence seq_1;
 (a && b) ##1 c;
endsequence

sequence seq_2;
  ##2 !d;
endsequence

property p;
  @(posedge clk) seq_1 |-> seq_2;
endpeoperty
a: assert property(p);
```

Property checks that, if the sequence seq_1 is true on a given positive edge of the clock, then start checking the seq_2 (“d” should be low, 2 clock cycles after seq_1 is true).

```
module asertion_ex;
  bit clk,a,b,c,d;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  sequence seq_1;
    (a && b) ##1 c;
  endsequence

  sequence seq_2;
    ##2 !d;
  endsequence
```

```
  property p;
    @(posedge clk) seq_1 |-> seq_2;
  endproperty
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

Timing windows in SVA Checkers

```
property p;
  @(posedge clk) a |-> ##[1:4] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then within 1 to 4 clock cycles, the signal "b" should be high.

Overlapping timing window

```
property p;
  @(posedge clk) a |-> ##[0:4] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" should be high in the same clock cycle or within 4 clock cycles.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[1:4] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[0:4] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

Indefinite timing window

- The upper limit of the timing window specified in the right-hand side can be defined with a "$" sign which implies that there is no upper bound for timing.
- This is called the "eventuality" operator.
- The checker will keep checking for a match until the end of the simulation.

```
property p;
  @(posedge clk) a |-> ##[1:$] b;
endproperty
a: assert property(p);
```

property checks that, if signal "a" is high on a given positive clock edge, then signal "b" will be high eventually starting from the next clock cycle.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##[1:$] b;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

SystemVerilog Repetition operators

```
property p;
  @(posedge clk) a |-> ##1 b ##1 b ##1 b;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 consecutive clock cycles.

The Consecutive repetition operator is used to specify that a signal or a sequence will match continuously for the number of clocks specified.

Syntax

```
signal [*n] or sequence [*n]

"n" is the number of repetitions.
```

with repetition operator above sequence can be re-written as,

```
property p;
  @(posedge clk) a |-> ##1 b[*3];
endproperty
a: assert property(p);
```

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##1 b[*3];
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

`a ##1 b [*3] ##1 c`	// Equiv. to **a ##1 b ##1 b ##1 b ##1 c**
`(a ##2 b) [*2]`	// Equiv. to **(a ##2 b ##1 a ##2 b)**

`(a ##2 b)[*1:3]`	// Equiv. to **(a ##2 b)**
	// or **(a ##2 b ##1 a ##2 b)**
	// or **(a ##2 b ##1 a ##2 b ##1 a ##2 b)**

go to repetition

The go-to repetition operator is used to specify that a signal will match the number of times specified not necessarily on continuous clock cycles.

```
signal [->n]
```

```
property p;
  @(posedge clk) a |-> ##1 b[->3] ##1 c;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 clock cycles followed by “c” should be high after ”b” is high for the third time.

```
module asertion_ex;
  bit clk,a,b,c;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) a |-> ##1 b[->3] ##1 c;
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

SVA Methods

$rose

```
$rose(boolean expression or signal name)
```

returns true if the least significant bit of the expression changed to 1. Otherwise, it returns false.

```
sequence seq_rose;
  @(posedge clk) $rose(a);
endsequence
```

Sequence seq_rose checks that the signal "a" transitions to a value of 1 on every positive edge of the clock. If the transition does not occur, the assertion will fail.

$fell

```
$fell(boolean expression or signal name)
```

Sequence seq_fell checks that the signal "a" transitions to a value of 0 on every positive edge of the clock. If the transition does not occur, the assertion will fail.

```
sequence seq_fell;
  @(posedge clk) $fell(a);
endsequence
```

Sequence seq_fell checks that the signal "a" transitions to a value of 0 on every positive edge of the clock. If the transition does not occur, the assertion will fai

$stable

```
$stable(boolean expression or signal name)
```

returns true if the value of the expression did not change. Otherwise, it returns false.

```
sequence seq_stable;
  @(posedge clk) $stable(a);
endsequence
```

Sequence seq_stable checks that the signal "a" is stable on every positive edge of the clock.
If there is any transition occurs, the assertion will fail.

$past

```
$past(signal_name, number of clock cycles)
```

provides the value of the signal from the previous clock cycle.

```
property p;
  @(posedge clk) b |-> ($past(a,2) == 1);
endproperty
a: assert property(p);
```

Property checks that, in the given positive clock edge, if the “b” is high, then 2 cycles before that, a was high.

```
module asertion_ex;
  bit clk,a,b;

  always #5 clk = ~clk; //clock generation

  //generating 'a'
  initial begin
        a=1; b=1;
    #15 a=0; b=0;
    #10 a=1; b=0;
    #10 a=0; b=0;
    #10 a=1; b=1;
    #10;
    $finish;
  end

  property p;
    @(posedge clk) b |-> ($past(a,2) == 1);
  endproperty

  //calling assert property
  a_1: assert property(p);

  //wave dump
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end
endmodule
```

$past construct with clock gating

The $past construct can be used with a gating signal. on a given clock edge, the gating signal has to be true even before checking for the consequent condition.

```
$past (signal_name, number of clock cycles, gating signal)
```

```
Property p;
  @(posedge clk) b |-> ($past(a,2,c) == 1);
endproperty
a: assert property(p);
```

Property checks that, in the given positive clock edge, if the “b” is high, then 2 cycles before that, a was high only if the gating signal "c' is valid on any given positive edge of the clock.

Built-in system functions

- $onehot(expression)
  - checks that only one bit of the expression can be high on any given clock edge.
- $onehot0(expression)
  - checks only one bit of the expression can be high or none of the bits can be high on any given clock edge.
- $isunknown(expression)
  - checks if any bit of the expression is X or Z.
- $countones(expression)
  - counts the number of bits that are high in a vector.

```
a_1: assert property( @(posedge clk) $onehot(state) );
a_2: assert property( @(posedge clk) $onehot0(state) );
a_3: assert property( @(posedge clk) $isunknown(bus) ) ;
a_4: assert property( @(posedge clk) $countones(bus)> 1 );
```

Assert statement a_1 checks that the bit vector "state" is one-hot.
Assert statement a_2 checks that the bit vector "state" is zero one-hot.
Assert statement a_3 checks if any bit of the vector "bus" is X or Z.
Assert statement a_4 checks that the number of ones in the vector "bus" is greater than one.

disable iff

In certain design conditions, we don't want to proceed with the check if some condition is true. this can be achieved by using disable iff.

```
property p;
  @(posedge clk)
  disable iff (reset) a |-> ##1 b[->3] ##1 c;
endproperty
a: assert property(p);
```

property checks that, if the signal “a” is high on given posedge of the clock, the signal “b” should be high for 3 clock cycles followed by “c” should be high after ”b” is high for the third time. During this entire sequence, if reset is detected high at any point, the checker will stop.

ended

while concatenating the sequences, the ending point of the sequence can be used as a synchronization point. This is expressed by attaching the keyword "ended" to a sequence name.

```
sequence seq_1;
 (a && b) ##1 c;
endsequence

sequence seq_2;
  d ##[4:6] e;
endsequence

property p;
  @(posedge clk) seq_1.ended |-> ##2 seq_2.ended;
endpeoperty
a: assert property(p);
```

property checks that, sequence seq_1 and SEQ_2 match with a delay of 2 clock cycles in between them. the end point of the sequences does the synchronization.

```
assert property(@(posedge clk)
   disable iff (!reset)
   a |=> b ##1 c);
```

when a gets the value 1 that b takes the value 1 one cycle later and that c takes the value 1 one additional cycle later or that reset got the value 0:

```
always_ff @(posedge clk)
   case (state)
    s0 :
      state <= s1;
      assert property
         ($past(state) = s4);
```

Here, the concurrent assertion is placed inside a case statement. The check of the assertion is performed every rising edge of the clock and is started only if the branch `s0` of the case statement is executed.

A simple UP/DOWN COUNTER design is presented. Counter assertions deployed directly at the source can greatly reduce the time to debug since these assertions will point to the exact cause of a Counter error without the need for extensive back-tracing debug when design fails.

A simple UP/DOWN COUNTER design is presented as the DUT.

*) The counter has 8 bit data input and 8 bit data output
*) When ld_cnt_ is asserted (active Low), data_in is loaded and output to data_out

*) When count_enb (active High) is enabled (high) and
    *) updn_cnt is high, data_out = data_out+1;
    *) updn_cnt is low, data_out = data_out-1;

*) When count_enb is LOW, data_out = data_out;

```
module counter (
  input clk, rst_, ld_cnt_, updn_cnt, count_enb,
  input [7:0] data_in,
  output logic [7:0] data_out
  );

always @ (posedge clk or negedge rst_)
begin
  if (!rst_)
    begin
    `ifndef check1
       data_out <= 0;
    `endif
     end              //LOAD DATA
  else                if (!ld_cnt_)
  begin                 data_out <= data_in;

                      //HOLD DATA
                      `ifndef check2
                       else if (!count_enb)
                        data_out <= data_out;
                      `endif
```

```
    //COUNT DATA
    `ifdef check3
    else
      case (updn_cnt)
        1'b1: data_out <= data_out - 1;
        1'b0: data_out <= data_out + 1;
      endcase
    `else
    else
      case (updn_cnt)
        1'b1: data_out <= data_out + 1;
        1'b0: data_out <= data_out - 1;
      endcase
    `endif

  end
end
endmodule
```

Code assertions to check for the following conditions in the 'counter' design.

CHECK # 1. Check that when 'rst_' is asserted (==0) that data_out == 8'b0

CHECK # 2. Check that if ld_cnt_ is deasserted (==1) and count_enb is not enabled (==0) that data_out HOLDS it's previous value.

Disable this property if rst is low.

CHECK # 3. Check that if ld_cnt_ is deasserted (==1) and count_enb is enabled (==1) that if updn_cnt==1 the count goes UP and if updn_cnt==0 the count goes DOWN.

Disable this property if rst is low.

```
//---------------------------------
//      CHECK # 1. Check that when 'rst_' is asserted (==0) that data_out == 8'b0
//---------------------------------
`ifdef check1
property counter_reset;
  @(clk) disable iff (rst_) !rst_ |=> (data_out == 8'b0);
endproperty

counter_reset_check: assert property(counter_reset)
  else $display($stime,,,"\t\tCOUNTER  RESET CHECK FAIL:: rst_=%b data_out=%0d \n",
                                                    rst_,data_out);
`endif
```

```
//-----------------------------------
//      CHECK # 2. Check that if ld_cnt_ is deasserted (==1) and count_enb is not enabled
//                 (==0) that data_out HOLDS it's previous value.
//                 Disable this property 'iff (!rst)'
//-----------------------------------
`ifdef check2
property counter_hold;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & !count_enb) |=> data_out ===
$past(data_out);
endproperty

counter_hold_check: assert property(counter_hold)
  else $display($stime,,,"\t\tCOUNTER  HOLD CHECK FAIL:: counter HOLD \n");
`endif
```

```
// CHECK # 3. Check that if ld_cnt_ is deasserted (==1) and count_enb is
// enabled (==1) that if updn_cnt==1 the count goes UP and if updn_cnt==0 the
// count goes DOWN.
//---------------------------------

`ifdef check3
property counter_count;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & count_enb) |->
    if (updn_cnt) ##1 (data_out-8'h01) == $past(data_out)
    else          ##1 (data_out+8'h01) == $past(data_out);
endproperty

counter_count_check: assert property(counter_count)
  else $display($stime,,,"\t\tCOUNTER COUNT CHECK FAIL:: UPDOWN COUNT using
$past \n");
```

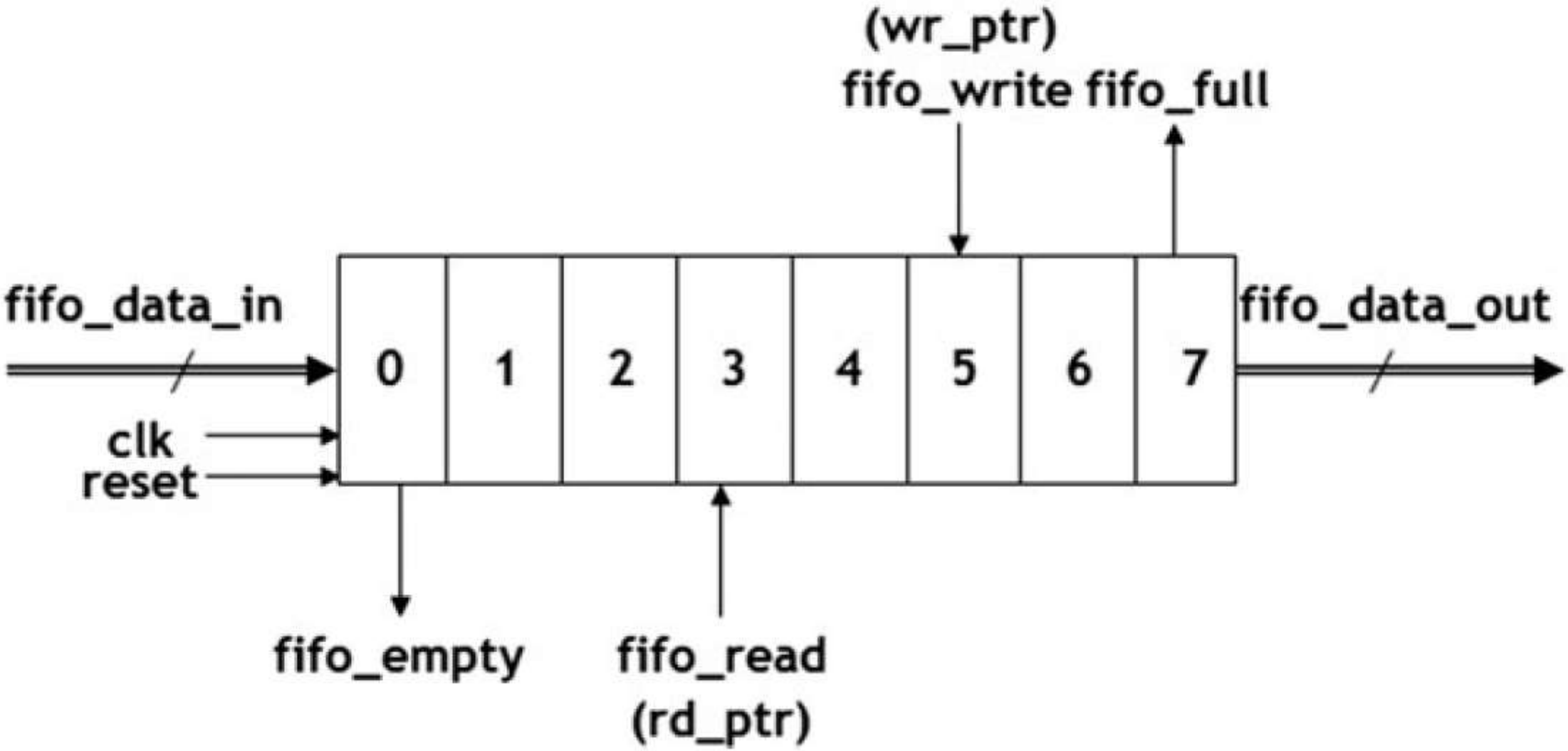
```
//-----------------------------------
// Alternate way of writing assertion for CHECK # 3
// Check for count using local variable
//-----------------------------------
/*
`ifdef check3
property counter_count_local;
logic[7:0] local_data;
  @(posedge clk) disable iff (!rst_) (ld_cnt_ & count_enb, local_data = data_out)
|->
    if (updn_cnt) ##1 (data_out == (local_data+8'h01))
    else          ##1 (data_out == (local_data-8'h01));
endproperty

counter_count_check: assert property(counter_count)
  else $display($stime,,,"\t\tCOUNTER COUNT CHECK FAIL:: UPDOWN COUNT using
$past \n");

`endif
*/
```

FIFO Specs

- fifo maintains a wr_ptr and a rd_ptr
    - wr_ptr increments by 1 everytime a write is posted to the fifo on a fifo_write request
    - rd_ptr increments by 1 everytime a read is posted to the fifo on a fifo_read request
- fifo maintains a 'cnt' that increments on a write and decrements on a read. It is used to signal fifo_full and fifo_empty conditions as follows
    - When fifo 'cnt' is >= 7, fifo_full is asserted
    - When fifo 'cnt' is 0, fifo_empty is asserted.


(wr_ptr)
fifo_write
fifo_full
fifo_data_in
0
1
2
3
4
5
6
7
fifo_data_out
clk
reset
fifo_empty
fifo_read
(rd_ptr)

FILES:

1. *fifo.v :: Verilog RTL for 'fifo'*

2. *fifo_property.sv :: SVA file for fifo assertion.s*
   This is the file in which you will add your assertions.

3. *test_fifo.sv :: Testbench for the fifo.*
   Note the use of 'bind' in this testbench.

```
module fifo (fifo_data_out,fifo_full,fifo_empty,fifo_write,fifo_read,clk,rst_,
             fifo_data_in);

parameter fifo_depth=8;
parameter fifo_width=8;

output logic [(fifo_width-1):0] fifo_data_out;
output logic fifo_full, fifo_empty;
input  logic fifo_write, fifo_read, clk, rst_;
input  logic [(fifo_width-1):0] fifo_data_in;

logic [(fifo_width-1):0] fifomem [0:(fifo_depth-1)];

logic [3:0] wr_ptr, rd_ptr;
logic [3:0] cnt;

 always @ (posedge clk or negedge rst_)
   if (!rst_) begin
     rd_ptr  <= 0;
     wr_ptr  <= 0;
     cnt   <= 0;
 `ifndef check1
     fifo_empty <= 1;
```

```
`endif
    fifo_full  <= 0;
  end
  else begin
    case ({fifo_write, fifo_read})
      2'b00: ;     // everyone's sleeping!
      2'b01: begin // read
               if (cnt>0) begin
                 rd_ptr <= rd_ptr + 1;
                 cnt  <= cnt  - 1;
               end
`ifdef check2
               if (cnt==0) fifo_empty <= 1;
`else
`ifdef check5
               if (cnt==1) fifo_empty <= 1;
             rd_ptr <= rd_ptr+1;
`else
               if (cnt==1) fifo_empty <= 1;
`endif
`endif
               fifo_full <= 0;
             end
```

Code assertions to check for the following conditions in the 'fifo' design.

CHECK # 1. Check that on reset
rd_ptr=0; wr_ptr=0; cnt=0; fifo_empty=1 and fifo_full=0

CHECK # 2. Check that fifo_empty is asserted when fifo 'cnt' is 0.
Disable this property 'iff (!rst)'

CHECK # 3. Check that fifo_full is asserted any time fifo 'cnt' is greater than 7.
Disable this property 'iff (!rst)'

CHECK # 4. Check that if fifo is full and you attempt to write (but not read) that the wr_ptr does not change.

CHECK # 5. Check that if fifo is empty and you attempt to read (but not write) that the rd_ptr does not change.

CHECK # 6. Write a property to Warn on write to a full fifo

CHECK # 7. Write a property to Warn on read from an empty fifo

```
//      ----------------------------------------
//        1. Check that on reset,
//                  rd_ptr=0; wr_ptr=0; cnt=0; fifo_empty=1 and fifo_full=0
//      ----------------------------------------


 `ifdef check1
property check_reset;
  @(posedge clk)
    (!rst_ |-> (`rd_ptr==0 && `wr_ptr==0 && fifo_empty==1 && fifo_full==0));
endproperty
check_resetP: assert property (check_reset) else $display($stime,"\t\t
FAIL::check_reset\n");
 `endif
```

```
//      ----------------------------------------
//        2. Check that fifo_empty is asserted the same clock that fifo 'cnt' is 0.
//          Disable this property 'iff (!rst)'
//      ----------------------------------------


    `ifdef check2
    property fifoempty;
      @(posedge clk) disable iff (!rst_)
        (`cnt==0 |-> fifo_empty);
    endproperty
    fifoemptyP: assert property (fifoempty) else $display($stime,"\t\t
    FAIL::fifo_empty condition\n");
    `endif
```

```
//     ----------------------------------------
//       3. Check that fifo_full is asserted any time fifo 'cnt' is greater than 7.
//          Disable this property 'iff (!rst)'
//     ----------------------------------------


    `ifdef check3
property fifofull;
  @(posedge clk) disable iff (!rst_)
    (`cnt>(fifo_depth-1)  |-> fifo_full);
endproperty
fifofullP: assert property (fifofull) else $display($stime,"\t\t FAIL::fifo_full
condition\n");
    `endif
```

```
//     ----------------------------------------
//       4. Check that if fifo is full and you attempt to write (but not read) that
//          the wr_ptr does not change.
//     ----------------------------------------



    `ifdef check4
    property fifo_full_write_stable_wrptr;
      @(posedge clk) disable iff (!rst_)
        (fifo_full && fifo_write && !fifo_read |=> $stable(`wr_ptr));
    endproperty
    fifo_full_write_stable_wrptrP:  assert property (fifo_full_write_stable_wrptr)
        else $display($stime,"\t\t FAIL::fifo_full_write_stable_wrptr  condition\n");
    `endif
```

```
//      ----------------------------------------
//        5. Check that if fifo is empty and you attempt to read (but not write) that
//           the rd_ptr does not change.
//      ----------------------------------------


  property fifo_empty_read_stable_rdptr;
    @(posedge clk) disable iff (!rst_)
      (fifo_empty && fifo_read && !fifo_write |=> $stable(`rd_ptr));
  endproperty
  fifo_empty_read_stable_rdptrP:  assert property (fifo_empty_read_stable_rdptr)
                  else $display($stime,"\t\t FAIL::fifo_empty_read_stable_rdptr
  condition\n");
  `endif
```

```
//       6. Write a property to Warn on write to a full fifo
//             This property will give Warning with all simulations
//      ----------------------------------------


`ifdef check6
property write_on_full_fifo;
  @(posedge clk) disable iff (!rst_)
    fifo_full |-> !fifo_write;
endproperty
write_on_full_fifoP: assert property (write_on_full_fifo)
                else $display($stime,"\t\t WARNING::write_on_full_fifo\n");
`endif
```

```
//     ----------------------------------------
//       7. Write a property to Warn on read from an empty fifo
//            This property will give Warning with all simulations
//     ----------------------------------------


`ifdef check7
property read_on_empty_fifo;
  @(posedge clk) disable iff (!rst_)
    fifo_empty |-> !fifo_read;
endproperty
read_on_empty_fifoP: assert property (read_on_empty_fifo)
          else $display($stime,"\t\t  WARNING::read_on_empty_fifo  condition\n");
`endif
```

```
module tb;
  bit a, b, c, d;
  bit clk;

  always #10 clk = ~clk;

  initial begin
    for (int i = 0; i < 20; i++) begin
      {a, b, c, d} = $random;
      $display("%0t a=%0d b=%0d c=%0d d=%0d", $time, a, b, c, d);
      @(posedge clk);
    end
    #10 $finish;
  end

  sequence s_ab;
    a ##1 b;
  endsequence

  sequence s_cd;
    c ##2 d;
  endsequence
```

```
  property p_expr;
    @(posedge clk) s_ab ##1 s_cd;
  endproperty

  assert property (p_expr);
endmodule
```

```
0 a=0 b=1 c=0 d=0
10 a=0 b=0 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 10ns failed at 10ns
        Offending 'a'
30 a=1 b=0 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 30ns failed at 30ns
        Offending 'a'
50 a=0 b=0 c=1 d=1
70 a=1 b=1 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 70ns failed at 70ns
        Offending 'a'
"testbench.sv", 28: tb.unnamed$$_3: started at 50ns failed at 70ns
        Offending 'b'
90 a=1 b=1 c=0 d=1
110 a=0 b=1 c=0 d=1
130 a=0 b=0 c=1 d=0
"testbench.sv", 28: tb.unnamed$$_3: started at 130ns failed at 130ns
        Offending 'a'
"testbench.sv", 28: tb.unnamed$$_3: started at 90ns failed at 130ns
        Offending 'c'
150 a=0 b=0 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 150ns failed at 150ns
        Offending 'a'
170 a=1 b=1 c=0 d=1
```

```
"testbench.sv", 28: tb.unnamed$$_3: started at 170ns failed at 170ns
        Offending 'a'
190 a=0 b=1 c=1 d=0
210 a=1 b=1 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 210ns failed at 210ns
        Offending 'a'
230 a=1 b=1 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 190ns failed at 230ns
        Offending 'c'
250 a=1 b=1 c=0 d=0
270 a=1 b=0 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 230ns failed at 270ns
        Offending 'c'
290 a=0 b=1 c=1 d=0
"testbench.sv", 28: tb.unnamed$$_3: started at 270ns failed at 290ns
        Offending 'b'
"testbench.sv", 28: tb.unnamed$$_3: started at 250ns failed at 290ns
        Offending 'c'
310 a=0 b=1 c=0 d=1
"testbench.sv", 28: tb.unnamed$$_3: started at 310ns failed at 310ns
        Offending 'a'
330 a=1 b=0 c=1 d=0
"testbench.sv", 28: tb.unnamed$$_3: started at 330ns failed at 330ns
        Offending 'a'
"testbench.sv", 28: tb.unnamed$$_3: started at 290ns failed at 330ns
        Offending 'c'
350 a=0 b=1 c=0 d=1
370 a=0 b=1 c=1 d=1
```

```
module tb;
    bit a, b;
    bit clk;

    always #10 clk = ~clk;

    initial begin
        for (int i = 0; i < 10; i++) begin
            @(posedge clk);
            a <= $random;
            b <= $random;
            $display("[%0t] a=%0b b=%0b", $time, a, b);
        end
        #10 $finish;
    end

  // This assertion runs for entire duration of simulation
  // Ensure that both signals are high at posedge clk
  assert property (@(posedge clk) a & b);

endmodule
```

Time (ns)	a	b	Result
10	0	0	FAIL
30	0	1	FAIL
50	1	1	PASS
70	1	1	PASS
90	1	0	FAIL
110	1	1	PASS
130	0	1	FAIL
150	1	0	FAIL
170	1	0	FAIL
190	1	0	FAIL